

Relatório de Evolução

Sprint 03 — Agente Conversacional com LangGraph

Disciplina: Prompt and Artificial Intelligence

Curso: Ciência da Computação — 1º ano

Parceiro: GoodWe Brasil

Turma: CCPW

Equipe

Guilherme Detta • Igor Alves • Henrique Bolfer • Arthur Costa • Rafaella Ferreira • Aneliza Rondina

Repositório de referência das Sprints 1–2: github.com/IgorLDev/chargegrid-chatbot

Objetivo da Sprint 03

Evoluir o chatbot para uma arquitetura de agente mais estruturada, com fluxo controlado pelo framework, memória por sessão, guardrails de segurança, testes formais e infraestrutura para comparação entre modelos.

STATUS DA ENTREGA

A implementação e a documentação estão organizadas. A única pendência externa é a execução experimental com créditos de API para obter métricas reais de comparação entre os dois LLMs.

1. Evolução: Sprints 1–2 → Sprint 03

Nas Sprints 1 e 2, o chatbot ChargeGrid Intelligence já utilizava LangGraph, porém em uma estrutura essencialmente linear. O fluxo principal era baseado em uma sequência de recuperação e geração, com histórico mantido manualmente em uma lista Python.

Na Sprint 03, a arquitetura foi aprofundada sem trocar o framework. O grafo passou a controlar etapas distintas de validação, recuperação, geração e proteção de saída. A memória passou a utilizar o mecanismo de checkpointer do LangGraph, associada a um `thread_id` por sessão.

Também foram adicionados guardrails independentes do modelo, uma suíte de testes de segurança e uma estrutura para comparar dois modelos de linguagem usando o mesmo conjunto de casos.

Fluxo atual

Entrada	Validação	Recuperação	LLM	Saída
Mensagem do usuário	Prompt Injection / escopo	Base de conhecimento	Resposta contextual	Guardrail de saída

2. Framework e decisões técnicas

Framework: LangGraph.

A escolha permite representar o agente como um grafo de estados, tornando explícitas as etapas e as decisões do fluxo. O mesmo framework também fornece o checkpointer utilizado para memória por sessão.

Vantagens observadas:

Aspecto	Decisão / benefício
Controle de fluxo	Nós separados para validação, recuperação, LLM e proteção de saída.
Memória	Checkpointer com <code>`thread_id`</code> , evitando histórico global compartilhado.
Flexibilidade de LLM	Configuração por chave permite alternar o provedor/modelo sem reescrever o agente.
Segurança	Guardrails no código complementam as instruções do system prompt.

Trade-offs: LangGraph exige mais estrutura do que uma chamada direta de API; o filtro por regex é determinístico, porém cobre principalmente padrões conhecidos; e a robustez final também depende do comportamento do modelo.

3. Comparativo antes x depois

Critério	Sprints 1–2	Sprint 03
Arquitetura	Fluxo linear baseado em `add_sequence`.	Grafo com validação, recuperação, LLM e guardrails.
Memória	Lista `chat_history` gerenciada pela aplicação.	Checkpointter do LangGraph por `thread_id`.
RAG	Recuperação de documentos.	RAG mantido como nó próprio, com suporte também a `.txt/.md`.
Segurança	Regras no system prompt.	System prompt + guardrails de entrada/saída + testes.
Modelos	Modelo fixo.	Dois modelos configurados e script de comparação.
Testes	Sem suíte formal equivalente documentada.	Testes funcionais, memória, segurança e comparação.

4. Métricas e estado dos experimentos

A infraestrutura para medir os modelos está implementada, mas a execução experimental completa não foi concluída por falta de créditos/quotas de API. Durante uma tentativa anterior, a API da OpenAI retornou erro de quota/créditos (`insufficient\_quota` / `credit\_balance\_exhausted`).

Métrica	Status
Latência por turno	Medição implementada; valor real depende de execução com API disponível.
Tamanho das respostas	Medição implementada; valor real depende de execução com API disponível.
Casos de segurança	Suíte implementada; resultados finais dependem de execução.
Memória em 3+ turnos	Teste automatizado implementado.
Comparação entre LLMs	Infraestrutura pronta; execução completa pendente de créditos.

Não foram incluídos números estimados ou inventados. Assim, o relatório preserva a rastreabilidade dos resultados e deixa clara a única dependência externa para completar a comparação quantitativa.

5. Problemas encontrados e soluções

Problema 1 — Isolamento de memória: havia o risco de compartilhar histórico entre sessões. **Solução:** utilizar o checkpointter do LangGraph com `thread\_id = session\_id`. O projeto possui teste específico para verificar o isolamento.

Problema 2 — Prompt Injection: as versões anteriores dependiam principalmente do system prompt. **Solução:** adicionar guardrail determinístico de entrada, combinado com as instruções do modelo, seguindo defesa em profundidade.

Problema 3 — Quota de API: a comparação experimental exigiu chamadas reais e uma execução encontrou falta de créditos. **Solução adotada:** não fabricar resultados; manter a infraestrutura pronta e registrar a limitação de forma explícita para posterior execução.

6. Segurança e testes

A solução inclui proteção contra prompt injection e controles de escopo. O agente também recebe instruções para não inventar especificações técnicas e para não fornecer aconselhamento jurídico/financeiro profissional ou instruções elétricas perigosas.

Categoria	Implementação
Prompt Injection	Detecção de padrões de manipulação antes da chamada ao LLM.
Escopo	Restrição ao contexto operacional do projeto GoodWe/ChargeGrid.
Especificações	Instrução explícita para não inventar dados técnicos.
Legal / financeiro	Redirecionamento em vez de aconselhamento profissional.
Segurança elétrica	Recusa de instruções perigosas e orientação para profissional qualificado.
Saída	Guardrail de saída para validar a resposta produzida.

7. Divisão da equipe

Integrante	RM	Responsabilidade
Guilherme Detta	569666	Organização dos requisitos e apoio na documentação dos testes funcionais.
Igor Alves	574127	Pesquisa de frameworks/modelos e apoio na comparação entre LLMs.
Henrique Bolfer	569514	Segurança, Prompt Injection e documentação dos guardrails.
Arthur Costa	569976	Revisão dos entregáveis, apresentação e preparação da demonstração.
Rafaella Ferreira	571030	Desenvolvimento principal: arquitetura LangGraph, LLMs, memória e organização do código.
Aneliza Rondina	572977	Documentação técnica, relatório de evolução e descrição da arquitetura/decisões.

8. Conclusão

A Sprint 03 transforma a implementação em uma arquitetura de agente mais estruturada: o fluxo passa a ser controlado pelo grafo, a memória é organizada por sessão, a segurança possui camadas próprias e os testes ficam formalizados. A comparação entre modelos está preparada para execução reproduzível.

A única informação que ainda não pode ser preenchida com honestidade sem uma nova execução é o resultado quantitativo da comparação entre os LLMs. Quando houver créditos de API, basta executar o script existente e registrar os valores reais.

Arquivos relacionados: `app/agent.py`, `app/memory.py`, `app/guardrails.py`, `tests/test\_memory.py`, `tests/test\_security.py`, `tests/run\_model\_comparison.py` e `docs/relatorio\_modelos.md`.